

PeriX – Reference Manual

Peridynamic Differential Operator (PDDO) Framework for Multiphysics Simulation

Yang Bai

Materials Mechanics Lab (MMLab)

yangbai90@outlook.com

Release tag: 2026-04-16, version 0.10

Abstract

PeriX is an open-source C++20 framework for solving steady and transient multiphysics problems by means of the *Peridynamic Differential Operator* (PDDO). The PDDO is a nonlocal collocation-style method due to E. Madenci and co-workers that reproduces ordinary partial derivatives of arbitrary order on a point cloud inside a finite *horizon* δ , using a weighted least-squares fit against a polynomial basis. PeriX ships with seven physics kernels – Poisson, transient diffusion, Cahn–Hilliard phase separation, implicit and explicit bond-breaking peridynamic fracture, and stress-assisted Cahn–Hilliard transport with and without fracture – driven entirely by a single JSON input file. This manual documents the public release: the build system, the command line, the complete JSON input schema with every recognised key, its type, default value and validation rule, the mathematical formulation of each kernel exactly as implemented, the boundary- and initial-condition machinery, the linear and nonlinear solvers, the output system, and all bundled example problems. Every statement in this manual has been cross-checked against the source code of the release.

Repository	https://github.com/MatMechLab/PeriX
License	GNU General Public License v3.0 (GPLv3)
Language	C++20 (CMake $\geq$ 3.21)
Input	a single JSON file (<code>perix -i input.json</code>)
Output	VTK <code>.vtu/.pvd</code> and/or ExodusII <code>.e</code>

Contents

1	Introduction	4
1.1	What is PeriX?	4
1.2	Scope of this manual	4
1.3	Source layout	4
2	Installation	5
2.1	Requirements	5
2.2	CMake options	5
2.3	Building	5
2.4	Running the test suite	5
3	Running PeriX	6
3.1	Command line	6
3.2	Files written by a run	6
4	The Peridynamic Differential Operator	6
4.1	Families, bonds and the horizon	6
4.2	Weight and partial-volume correction	7
4.3	Moment matrix and operator weights	7
4.4	Cracks and modified families	7
5	Meshes and the PD Point Cloud	7
5.1	Built-in structured mesh ("type": "perix")	8
5.2	Imported Gmsh mesh ("type": "gmsh")	8
5.3	Built-in disc mesh ("type": "circle")	8
5.4	Ghost boundary layer	8
5.5	Variable horizon	8
5.6	Pre-existing cracks (MeshModify)	8
6	JSON Input Reference	9
6.1	General structure and validation	9
6.2	Mesh	9
6.3	PDMesh	10
6.4	MeshModify	10
6.5	DOFs	10
6.6	Cross-block rules	11
7	Physics Kernels	11
7.1	The ElmtSystem block	11
7.2	poisson – steady Poisson equation	12
7.3	diffusion – transient diffusion	12
7.4	cahnhilliard – phase separation	12
7.5	Bond-based elasticity and fracture: common ground	13
7.6	pddo_dynamic_frac – implicit dynamic fracture	14
7.7	explicit_pddo_frac – explicit dynamic fracture	14
7.8	stress_cahnhilliard – chemo-mechanics	15
7.9	frac_stress_cahnhilliard – chemo-mechanics with fracture	15

8	Boundary Conditions	16
8.1	The BCSys <code>tem</code> block	16
8.2	<code>dirichlet</code> – essential conditions	16
8.3	<code>neumann</code> – homogeneous natural conditions	17
8.4	<code>speciesflux</code> – prescribed species influx	17
8.5	<code>pdtraction</code> and <code>sdtraction</code> – strong-form tractions	17
9	Initial Conditions	18
10	Job Control, Time Stepping and Solvers	18
10.1	<code>JobSystem</code>	18
10.2	Run drivers	18
10.3	<code>TimeStepping</code>	19
10.4	<code>NonlinearSolver</code>	19
10.5	<code>LinearSolver</code>	19
11	Output System	20
11.1	The <code>OutputSystem</code> block	20
11.2	VTK output	20
11.3	ExodusII output	20
12	Worked Examples	21
12.1	<code>poisson.json</code> – steady conduction	21
12.2	<code>poisson_amgcl.json</code> and <code>poisson_gpu.json</code>	21
12.3	<code>diffusion.json</code> – transient diffusion	21
12.4	<code>spinodal.json</code> – Cahn–Hilliard phase separation	21
12.5	<code>kalthoff_winkler.json</code> – explicit dynamic fracture	22
12.6	<code>kalthoff_winkler_3D.json</code> – 3-D explicit fracture	22
12.7	<code>tensile_plate.json</code> – implicit quasi-static fracture	22
12.8	<code>silicon.json</code> – lithiation of a silicon disc	22
12.9	<code>silicon_frac.json</code> – lithiation with a pre-crack	22
13	Testing	23
14	License and Citation	23

1 Introduction

1.1 What is PeriX?

PeriX discretises partial differential equations on a *peridynamic point cloud*: every material point i carries a position $\mathbf{x}_i$, a volume V_i , and a family $\mathcal{H}_i$ of neighbouring points within a horizon δ . Spatial derivatives are replaced by the Peridynamic Differential Operator, a set of per-bond weights $g_i^q(\mathbf{x}_j)$ such that, for a smooth field u ,

$$\left. \frac{\partial^{|q|} u}{\partial x^{q_1} \partial y^{q_2} \partial z^{q_3}} \right|_{\mathbf{x}_i} \approx \sum_{j \in \mathcal{H}_i} g_i^q(\mathbf{x}_j) (u_j - u_i), \quad (1)$$

where the neighbour volume V_j is already contained in g_i^q . Because (1) needs no mesh topology, no shape functions and no integration rule, the same machinery serves diffusion-type equations, fourth-order Cahn–Hilliard systems, and bond-based dynamic fracture on one code path.

The public release contains seven ready-to-use physics kernels (Table 1); each is selected by a single type string in the `ElmtSystem` block of the input file.

Table 1: Physics kernels in the public release.

Kernel type	DoFs	Physics
poisson	u	steady Poisson equation $\sigma \nabla^2 u + f = 0$
diffusion	c	transient diffusion $\dot{c} = D \nabla^2 c + f$ (backward Euler)
cahnhilliard	c,mu	Cahn–Hilliard phase separation, mixed two-field split
pddo_dynamic_frac	ux,uy,(uz)	implicit PDDO elastodynamics with critical-stretch bond failure
explicit_pddo_frac	ux,uy,(uz)	explicit (central-difference) matrix-free variant of the above
stress_cahnhilliard	c,mu,ux,uy,(uz)	stress-assisted Cahn–Hilliard transport coupled to elasticity
frac_stress_cahnhilliard	c,mu,ux,uy,(uz)	stress-assisted CH with bond-breaking fracture

1.2 Scope of this manual

This manual documents exactly what the public repository accepts and computes. The JSON validator (`src/InputSystem/InputSchema.cpp`) is strict: any unknown block, unknown key, wrong type or out-of-range value aborts the run with a descriptive error message before any computation starts. Consequently, every key listed in Section 6 is the complete public vocabulary – there are no hidden options.

1.3 Source layout

Table 2: Repository layout.

Directory	Contents
<code>src/, include/</code>	C++ sources and headers, one directory per subsystem (<code>Mesh</code> , <code>PDMesh</code> , <code>ElmtSystem</code> , <code>BCSystem</code> , <code>ICSystem</code> , <code>EquationSystem</code> , <code>LinearSolver</code> , <code>NonlinearSolver</code> , <code>TimeStepping</code> , <code>OutputSystem</code> , <code>InputSystem</code> , <code>PDProblem</code> , <code>Utils</code>)
<code>external/</code>	vendored header-only dependencies: Eigen and nlohmann/json (no downloads at configure time)
<code>examples/</code>	eleven ready-to-run JSON decks plus Gmsh meshes (Section 12)
<code>tests/</code>	unit and integration tests registered with CTest
<code>documents/</code>	this manual

2 Installation

2.1 Requirements

- CMake ≥ 3.21 ;
 - a C++20 compiler (GCC, Clang or MSVC);
 - nothing else for the base build – Eigen and nlohmann/json are vendored in `external/`.
- Optional features and their extra requirements:
- **OpenMP assembly** – an OpenMP-capable compiler (enabled by default, see below);
 - **Intel MKL PARDISO** direct solver – an Intel oneAPI or standalone MKL installation;
 - **AMGCL** iterative solver – an AMGCL source tree and OpenMP;
 - **CUDA assembly** – the CUDA toolkit;
 - **NVIDIA cuDSS** direct solver – the cuDSS library and the CUDA toolkit.

2.2 CMake options

Table 3: CMake configure options. “Compile definition” is the preprocessor symbol the option enables in the code.

Option (default)	Compile definition	Effect / additional variables
PARALLEL_ASSEMBLE (ON)	PERIX_PARALLEL_ASSEMBLE	builds the OpenMP assembly backend; needed for "assemble": "openmp"
USE_ONEAPI (OFF)	PERIX_HAS_PARDISO	builds the MKL PARDISO backend; set ONEAPI_DIR (or the MKLROOT environment variable) to the oneAPI/MKL root
USE_AMGCL (OFF)	PERIX_HAS_AMGCL	builds the AMGCL CPU/OpenMP backend; set AMGCL_DIR to the AMGCL source or install root; compiled with AMGCL_NO_BOOST; requires OpenMP
CUDA_ASSEMBLE (OFF)	PERIX_CUDA_ASSEMBLE	builds the CUDA assembly backend; needed for "assemble": "cuda"
USE_CUDSS (OFF)	PERIX_HAS_CUDSS	builds the NVIDIA cuDSS backend; set CUDSS_DIR to the cuDSS root (and CUDA_DIR if the toolkit is not in a default location)

2.3 Building

```
git clone https://github.com/MatMechLab/PeriX.git
cd PeriX
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

The executable is `build/perix`; it links against the static library `perix_core` that contains the entire framework. A fully featured configure line looks like:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DUSE_ONEAPI=ON -DONEAPI_DIR=/opt/intel/oneapi \
  -DUSE_AMGCL=ON -DAMGCL_DIR=$HOME/amgcl \
  -DCUDA_ASSEMBLE=ON -DUSE_CUDSS=ON -DCUDSS_DIR=/opt/nvidia/cudss
```

2.4 Running the test suite

The repository registers 29 CTest cases grouped under the labels `unit`, `integration`, `examples` and `amgcl`:

```
cd build
ctest -j 16 --output-on-failure # everything
ctest -L unit # only the unit tests
ctest -L examples # run the bundled decks
```

3 Running PeriX

3.1 Command line

```
perix -i input.json # parse, validate and solve
perix -i input.json --read-only # parse and validate only, no solve
```

The command line accepts exactly two options:

-i <file> the JSON input file. The name *must* end in `.json`; anything else is rejected.
--read-only stop after reading and validating the input file. Useful to syntax-check a deck without running it.

Any parse or validation error prints a descriptive message and the process exits with a non-zero status. On startup PeriX prints a banner with the version (0.10) and release date (2026-04-16).

3.2 Files written by a run

For an input file `job.json` (“stem” = `job`):

Table 4: Output files produced for input `job.json`.

File	Contents
<code>job-pdmesh.vtu</code>	the PD point cloud (always written after mesh generation)
<code>job-mesh.vtu</code>	the background mesh; only when <code>"savemesh": true</code> in the <code>Mesh</code> block
<code>job-PD-results.vtu</code>	static analysis: nodal solution and projected fields on the PD points
<code>job-PD-results-00000.vtu, ...</code>	transient analysis: one file per saved step
<code>job-PD-results.pvd</code>	ParaView time-series index over the PD result files
<code>job-Element-results[-NNNNN].vtu</code>	cell-data view of the same results on the background mesh, plus <code>job-Element-results.pvd</code>
<code>job.e</code>	ExodusII database; only when <code>"format"</code> is <code>"exodus"</code> or <code>"both"</code>

Transient runs save step 0, every `interval`-th step, and the final step (Section 11).

4 The Peridynamic Differential Operator

This section states the PDDO construction exactly as implemented in `src/PDMesh/CalcPDOperators.cpp`.

4.1 Families, bonds and the horizon

Every PD point i owns a family $\mathcal{H}_i = \{j \neq i : |\boldsymbol{\xi}_{ij}| \leq \delta_i\}$ with $\boldsymbol{\xi}_{ij} = \mathbf{x}_j - \mathbf{x}_i$ the *bond* vector. The horizon is

$$\delta = m \cdot \Delta x, \quad (2)$$

where m is the input key `HorizonRadiusFactor` and Δx the characteristic grid spacing of the mesh. With `VariableHorizon` enabled the per-point horizon becomes $\delta_i = \delta s_i / s_{\text{ref}}$ (Section 5.5).

4.2 Weight and partial-volume correction

Each bond carries a Gaussian influence weight

$$w(|\boldsymbol{\xi}|) = \exp \left[- \left(\frac{2|\boldsymbol{\xi}|}{\delta} \right)^2 \right], \quad (3)$$

and a Madenci-style partial-volume correction factor

$$v_c(|\boldsymbol{\xi}|) = \begin{cases} 1, & |\boldsymbol{\xi}| \leq \delta - \Delta x/2, \\ \frac{\delta - |\boldsymbol{\xi}| + \Delta x/2}{\Delta x} \text{ clamped to } [0, 1], & \text{otherwise,} \end{cases} \quad (4)$$

which linearly fades the volume of neighbours whose cells straddle the horizon sphere. The corrected neighbour volume $v_c V_j$ is used everywhere below.

4.3 Moment matrix and operator weights

Let N be the polynomial order (input key `Order`, minimum 2, maximum 6). The monomial basis on the *normalised* bond $\boldsymbol{\eta} = \boldsymbol{\xi}/\delta$ contains every multi-index $q = (q_1, q_2, q_3)$ with $1 \leq q_1 + q_2 + q_3 \leq N$ ($q_3 = 0$ in 2-D); the constant term is excluded because the operator acts on differences $u_j - u_i$. For each point i the symmetric moment matrix

$$A_{ab} = \sum_{j \in \mathcal{H}_i} w v_c V_j \boldsymbol{\eta}^{q_a} \boldsymbol{\eta}^{q_b} \quad (5)$$

is assembled, where $\boldsymbol{\eta}^q = \eta_x^{q_1} \eta_y^{q_2} \eta_z^{q_3}$. The coefficient matrix a solves

$$A a = B, \quad B_{ab} = q_1! q_2! q_3! \delta_{ab}, \quad (6)$$

via a full-pivot LU decomposition; the solution is accepted only if the relative residual is below 10^{-8} , otherwise the point is reported as degenerate and the run aborts (with one exception, see Section 5.4). Finally the operator weight of bond (i, j) for derivative q_k is

$$g_i^{q_k}(\mathbf{x}_j) = \frac{w v_c V_j}{\delta^{|q_k|}} \sum_a a_{ak} \boldsymbol{\eta}^{q_a}, \quad (7)$$

and any partial derivative of a field u is evaluated by the sum (1). Note that V_j is baked into $g_i^{q_k}$ – kernels never multiply by the neighbour volume again.

Operators are stored under human-readable names: `d/dx`, `d/dy`, `d2/dx2`, `d2/dxdy`, `d2/dy2`, `d3/dx3`, ... – one per multi-index up to order N . Order $N = 2$ (the minimum the validator accepts) already provides all first and second derivatives, which is what every bundled kernel consumes; higher orders increase the accuracy of those derivatives and make higher derivatives available.

4.4 Cracks and modified families

When `MeshModify` defines pre-existing cracks (Section 5.6), bonds that geometrically intersect a crack segment are excluded from both the moment matrix (5) and the operator sums (7) – *unless* the treatment is `force_only`, in which case the bond stays in the family and the operators, and the fracture kernels instead start it with zero mechanical health (Section 7.5).

5 Meshes and the PD Point Cloud

PeriX first builds (or imports) a background mesh, then collapses every cell to one PD point at its centroid carrying the cell measure as its volume V_i . All physics lives on the PD cloud; the background mesh is kept only for visualisation (`*-Element-results*.vtu`).

5.1 Built-in structured mesh ("type": "perix")

Generates a uniform grid of `quad4` cells in 2-D or `hex8` cells in 3-D on the box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] (\times [z_{\min}, z_{\max}])$ with $n_x \times n_y (\times n_z)$ cells. PD points sit at cell centres and carry $V_i = \Delta X \Delta Y (\Delta Z)$.

5.2 Imported Gmsh mesh ("type": "gmsh")

Reads an *ASCII* Gmsh MSH 4.1 file (binary files are rejected). Supported element types are POINT, EDGE2, TRI3, QUAD4, TET4 and HEX8; mixing different element types of the top dimension in one file is rejected. The file must contain a `$PhysicalNames` section – physical groups become PeriX node groups usable in boundary conditions. PD points sit at cell centroids with V_i the cell area/volume; the characteristic spacing used for the horizon is the median of $V_i^{1/\dim}$.

5.3 Built-in disc mesh ("type": "circle")

Generates a 2-D disc of given radius as `layers` concentric rings of cells around the centre; ring thickness (and hence local point density) is uniform in radius. The outer rim is wrapped in ghost points when `boundary` is `true` (or the string "outer"), producing the groups `outernodes` (bulk rim) and `outernodes_ghost` (fictitious layer).

5.4 Ghost boundary layer

Boundary conditions in a nonlocal method need a *fictitious layer* of points outside the physical domain. PeriX adds `ghost_layer` rings (default 1) of ghost points on the outer boundary:

- structured meshes: ghosts are placed half a cell outside each boundary face;
- imported meshes: ghosts are the mirror images of the near-boundary bulk points, reflected across the boundary face plane;
- disc meshes: ghosts continue the outermost ring pattern.

Each boundary wall `<wall>` yields two node groups: `<wall>nodes_ghost` (the ghost points) and `<wall>nodes` (their mirror bulk partners). Structured walls are named `left`, `right`, `bottom`, `top` (plus `back`, `front` in 3-D); e.g. `leftnodes_ghost`. Imported meshes keep their Gmsh physical names and the same `_ghost` suffix convention. The group `alldomain` always addresses every bulk (non-ghost) point.

Ghost points participate in neighbour families and PDDO operators like any other point; their rows in the global system are what the boundary conditions overwrite (Section 8). Points in ghost layers deeper than the first that end up with a degenerate moment matrix receive zero operators instead of aborting the run (their rows are fully replaced by BCs anyway); a degenerate *bulk* point is always a fatal error.

5.5 Variable horizon

With `"VariableHorizon": true` in `PDMesh`, each point gets its own horizon

$$\delta_i = \delta \frac{s_i}{s_{\text{ref}}}, \quad s_i = V_i^{1/\dim}, \quad s_{\text{ref}} = \text{median}_k s_k, \quad (8)$$

so the physical horizon scales with the local point spacing on graded meshes (e.g. the `circle` mesh). A pair (i, j) is bonded if $|\xi_{ij}| \leq \max(\delta_i, \delta_j)$, keeping bonds symmetric. Fracture kernels then use the pairwise mean $\delta_b = \frac{1}{2}(\delta_i + \delta_j)$ in the critical-stretch formula.

5.6 Pre-existing cracks (`MeshModify`)

The `MeshModify` block (allowed only together with a fracture kernel) cuts cracks into the initial bond network. Cracks can be given as explicit segments (`Cracks`) or generated from parametric

presets (Presets, types `center_crack` and `edge_crack`); Section 6.4 lists every key. With the only public treatment, `force_only`, severed bonds keep contributing to the PDDO operators (so gradients stay accurate near the crack faces) while the fracture kernels start them at zero mechanical health – the crack is invisible to transport terms but mechanically open from step one.

6 JSON Input Reference

6.1 General structure and validation

The input is a single JSON object. Twelve top-level blocks are recognised; any other key is rejected. Validation is strict and happens before any computation: unknown keys *inside* a block, wrong JSON types, and out-of-range values all abort the run with a message naming the offending block and key.

Table 5: Top-level blocks.

Block	Presence	Purpose
Mesh	required	background mesh (Section 6.2)
PDMesh	required	horizon, PDDO order, ghost layers (Section 6.3)
MeshModify	optional	pre-existing cracks (Section 6.4)
DOFs	required	symbolic field names (Section 6.5)
ElmtSystem	required	the physics kernel and its parameters (Section 7)
BCSystem	required	boundary conditions (Section 8)
ICSystem	optional	initial conditions (Section 9)
JobSystem	optional	static/transient switch, assembly backend (Section 10.1)
TimeStepping	see text	time step control; required for transient jobs, forbidden for static ones
NonlinearSolver	optional	Newton iteration control (Section 10.4)
LinearSolver	optional	linear solver selection (Section 10.5)
OutputSystem	optional	output cadence, fields, format (Section 11)

With `--read-only` (or when only the mesh is being inspected) just the `Mesh` block is mandatory; a normal solve requires `Mesh`, `PDMesh`, `DOFs`, `ElmtSystem` and `BCSystem`.

6.2 Mesh

The mandatory key `type` selects one of three generators: `"perix"` (structured box), `"gmsh"` (import), or `"circle"` (2-D disc).

"type": "perix" – structured box

Table 6: Mesh keys for `type=perix`.

Key	Type	Default	Meaning / constraints
<code>dim</code>	int	required	2 or 3
<code>meshtype</code>	string	required	"quad4" in 2-D, "hex8" in 3-D (enforced)
<code>nx,ny</code>	int	required	cells per direction, ≥ 1
<code>nz</code>	int	required in 3-D	rejected in 2-D, as are <code>zmin/zmax</code>
<code>xmin,xmax</code>	number	0, 1	domain extent; every minimum must be $<$ its maximum
<code>ymin,ymax</code>	number	0, 1	
<code>zmin,zmax</code>	number	0, 1	3-D only
<code>savemesh</code>	bool	false	write <code><stem>-mesh.vtu</code>

Table 7: Mesh keys for type=gms.

Key	Type	Default	Meaning / constraints
file	string	required	path to an <i>ASCII</i> Gmsh MSH 4.1 file; binary files are rejected
savemesh	bool	false	write <stem>-mesh.vtu

"type": "gms" – imported mesh

The file must contain `$PhysicalNames`; the physical groups become the node groups referenced by boundary conditions. Supported cell types: POINT, EDGE2, TRI3, QUAD4, TET4, HEX8; mixing different cell types of the highest dimension is rejected.

"type": "circle" – 2-D disc

Table 8: Mesh keys for type=circle.

Key	Type	Default	Meaning / constraints
radius	number	required	disc radius, > 0
layers	int	required	number of concentric point rings, ≥ 1
center	array	origin	2 or 3 numbers
boundary	bool or string	true	true : wrap the rim in ghost points with group base name outer ; a non-empty string sets a custom base name; false : no ghost rim
savemesh	bool	false	write <stem>-mesh.vtu

The rim groups are <base>nodes (outermost bulk ring) and <base>nodes_ghost (fictitious rim), i.e. outernodes / outernodes_ghost by default.

6.3 PDMesh

Table 9: PDMesh keys.

Key	Type	Default	Meaning / constraints
HorizonRadiusFactor	number	required	m in $\delta = m \Delta x$; must be positive
Order	int	required	PDDO polynomial order N ; the validator accepts $2 \leq N \leq 6$ (the published kernels require at least 2)
VariableHorizon	bool	false	per-point horizon scaling for graded meshes (Section 5.5)
ghost_layer	int	1	number of fictitious boundary layers, ≥ 1

A useful rule of thumb is $m \approx 3$ with Order 2; the bundled examples use horizon factors between 2.03 and 3.03.

6.4 MeshModify

Allowed only when the element is one of the fracture kernels.

At least one of Cracks / Presets must be present. Each Cracks item is an object {x1, y1, x2, y2} (numbers, required; the two endpoints must differ) plus an optional label string.

Each Presets item has type "center_crack" or "edge_crack":

6.5 DOFs

A non-empty array of unique, non-empty field-name strings, e.g. ["c", "mu", "ux", "uy"]. The names are *not* free: every kernel prescribes its canonical list and order (Table 1); the validator rejects any deviation. Boundary and initial conditions refer to these names in their dofs/dof keys.

Table 10: MeshModify keys.

Key	Type	Default	Meaning / constraints
type	string	required	must be "pre_existing_cracks"
treatment	string	required	must be "force_only": severed bonds stay in the PDDO operators but start with zero mechanical health
Cracks	array	optional	explicit 2-D crack segments, see below
Presets	array	optional	parametric crack generators, see below

Table 11: Presets item keys.

Key	Applies to	Meaning / constraints
center or center_x/center_y/center_z	center_crack	crack midpoint; the array form and the per-component form are mutually exclusive; components default to the domain centre
length	both	crack length, > 0 ; required
width	both	out-of-plane extrusion width (3-D), > 0 , optional
angle_degrees or angle_radians	both	in-plane inclination; mutually exclusive; edge_crack defaults to the inward normal of its side (0 for left , π for right , $\pi/2$ for bottom , $-\pi/2$ for top)
normal, axis	center_crack	optional 3-vectors selecting the crack-plane normal / in-plane axis in 3-D; without them (and without width) a 3-D centre crack is extruded through the whole thickness
side	edge_crack	required: "left", "right", "bottom" or "top"
position	edge_crack	coordinate of the mouth along the chosen side
label	both	optional name echoed in the log

6.6 Cross-block rules

After all blocks are validated individually, the following global rules are enforced (each quotes the actual error condition):

- a transient job requires a TimeStepping block; a static job must not have one;
- explicit_pddo_frac requires a transient job;
- explicit_pddo_frac is matrix-free and rejects both LinearSolver and NonlinearSolver blocks, and also pdtraction boundary conditions (they need an assembled system);
- "field": "velocity" initial conditions are accepted only with the explicit central-difference element;
- speciesflux boundary conditions require a species transport kernel (cahnhilliard, stress_cahnhilliard or frac_stress_cahnhilliard);
- MeshModify requires a fracture kernel (pddo_dynamic_frac, explicit_pddo_frac or frac_stress_cahnhilliard).

7 Physics Kernels

7.1 The ElmtSystem block

ElmtSystem is an object containing *exactly one* entry. The entry's key is a free label (it only appears in error messages); its value is an object with the two keys type (one of the seven kernel names) and params (an object, always required):

```
"ElmtSystem": {
  "myblock": {
    "type": "poisson",
    "params": {"sigma": 1.0, "f": 1.0 }
  }
}
```

```
}
}
```

The top-level `DOfs` array must match the kernel’s canonical field list of Table 1 exactly – same names, same order. For the mechanical kernels on a mesh whose dimension cannot be determined at validation time (an imported Gmsh file), both the 2-D and the 3-D DoF lists are accepted and the actual mesh decides.

Every kernel exposes *projections*: derived nodal fields (gradients, stresses, damage, ...) computed after each converged step and written to the output files when requested by name in `OutputSystem.Fields` (Section 11).

7.2 poisson – steady Poisson equation

Solves

$$\sigma \nabla^2 u + f = 0 \quad (9)$$

on the PD cloud, with the Laplacian evaluated through the PDDO second-derivative operators ($\nabla^2 = \text{d2/dx2} + \text{d2/dy2} + \text{d2/dz2}$). The kernel is linear; a static job converges in one Newton iteration.

Table 12: poisson parameters.

Key	Type	Constraint	Meaning
sigma	number	> 0, required	conductivity σ
f	number	required	constant source term f

Projections: `gradient` = ∇u and `flux` = $-\sigma \nabla u$ (vector fields).

7.3 diffusion – transient diffusion

Solves

$$\frac{\partial c}{\partial t} = D \nabla^2 c + f \quad (10)$$

with a backward-Euler residual $R_i = (c_i - c_i^{\text{old}})/\Delta t - D \nabla^2 c_i - f$. The kernel is transient-only: it refuses to run without a positive time step, so the job must be `transient` with a `TimeStepping` block.

Table 13: diffusion parameters.

Key	Type	Constraint	Meaning
D	number	> 0, required	diffusivity
f	number	required	constant source term

Projections: `gradient` = ∇c , `flux` = $-D \nabla c$.

7.4 cahnhilliard – phase separation

Solves the Cahn–Hilliard equation in the mixed two-field form

$$\frac{\partial c}{\partial t} = \nabla \cdot [M(c) \nabla \mu], \quad (11)$$

$$\mu = f'(c) - \kappa \nabla^2 c, \quad (12)$$

with the regular-solution free energy

$$f'(c) = \ln \frac{c}{1-c} + \chi(1-2c), \quad f''(c) = \frac{1}{c(1-c)} - 2\chi, \quad (13)$$

and the degenerate mobility $M(c) = M_0 c(1 - c)$. The concentration is clamped to $[\varepsilon, 1 - \varepsilon]$ with $\varepsilon = 10^{-8}$ inside the logarithm and the mobility, so the scheme survives $c \rightarrow 0, 1$. The mobility is Picard-frozen: it is evaluated at the current iterate and treated as constant in the Jacobian.

Three implementation details matter for accuracy and mass conservation:

- **Conservative flux.** The transport term is discretised with the antisymmetrised bond weight

$$w_{ij} = \frac{1}{2} \left[M_i g_{ij}^2 + \frac{V_j}{V_i} M_j g_{ji}^2 \right], \quad \nabla \cdot (M \nabla \mu)|_i \approx \sum_j w_{ij} (\mu_j - \mu_i), \quad (14)$$

where g_{ij}^2 is the PDDO Laplacian weight of bond (i, j) . This makes the exchange between i and j exactly antisymmetric after multiplication by V_i , so the total mass $\sum_i V_i c_i$ is conserved to solver tolerance.

- **No flux through ghosts.** Bonds that reach into the fictitious boundary layer are dropped from the species flux and from the $\kappa \nabla^2 c$ term, closing the domain (zero-flux walls) regardless of the boundary conditions applied to the ghost rows.
- **Laplacian calibration.** Each node's Laplacian weights are rescaled by $\lambda_i = 2 \dim / \sum_j g_{ij}^2 |\mathbf{x}_j - \mathbf{x}_i|^2$ (the exact Laplacian of $|\mathbf{x}|^2$ is $2 \dim$), which repairs the one-sided stencils next to the boundary.

Table 14: `cahnhilliard` parameters.

Key	Type	Constraint	Meaning
<code>chi</code>	number	required	Flory interaction parameter χ
<code>kappa</code>	number	> 0 , required	gradient-energy coefficient κ
<code>M</code>	number	> 0 , required	mobility scale M_0 in $M(c) = M_0 c(1 - c)$

Projections: `fPrime` = $f'(c)$ and `fDoublePrime` = $f''(c)$ (scalars).

7.5 Bond-based elasticity and fracture: common ground

The four mechanical kernels share one constitutive core. With displacement vector $\mathbf{u}$ and the isotropic stiffness constants

$$\begin{aligned} \text{plane stress: } C_{11} &= \frac{E}{1 - \nu^2}, \quad C_{12} = \nu C_{11}, \quad C_{66} = \frac{E}{2(1 + \nu)}, \\ \text{plane strain / 3-D: } C_{11} &= \lambda + 2\mu_s, \quad C_{12} = \lambda, \quad C_{66} = \mu_s, \end{aligned} \quad (15)$$

(λ, μ_s the Lamé constants), the discrete Navier operator at point i is assembled bond-wise from the PDDO second-derivative weights $G_{ij}^{xx}, G_{ij}^{yy}, G_{ij}^{xy}, \dots$. In 2-D:

$$\begin{aligned} L_x|_i &= \sum_j \mu_{ij} \left[(C_{11} G_{ij}^{xx} + C_{66} G_{ij}^{yy}) \Delta u_x + (C_{12} + C_{66}) G_{ij}^{xy} \Delta u_y \right], \\ L_y|_i &= \sum_j \mu_{ij} \left[(C_{12} + C_{66}) G_{ij}^{xy} \Delta u_x + (C_{66} G_{ij}^{xx} + C_{11} G_{ij}^{yy}) \Delta u_y \right], \end{aligned} \quad (16)$$

with $\Delta u_\bullet = u_{\bullet j} - u_{\bullet i}$ and the same pattern extended in 3-D. Here $\mu_{ij} \in \{0, 1\}$ is the *bond health*: 1 while intact, 0 once broken.

Critical stretch. A bond breaks when its stretch $s = (|\boldsymbol{\xi} + \boldsymbol{\eta}_u| - |\boldsymbol{\xi}|)/|\boldsymbol{\xi}|$ (with $\boldsymbol{\eta}_u$ the relative displacement) exceeds the critical value calibrated from the energy release rate G_c :

$$s_{cr} = \begin{cases} \sqrt{\frac{4\pi G_c}{9E\delta}} & \text{plane stress,} \\ \sqrt{\frac{5\pi G_c}{12E\delta}} & \text{plane strain,} \\ \sqrt{\frac{5G_c}{6E\delta}} & \text{3-D,} \end{cases} \quad (17)$$

With `"tension_only": true` only tensile stretches ($s > s_{cr}$) break bonds; with `false` compressive bonds break too ($|s| > s_{cr}$). Bond failure is irreversible. With variable horizons the bond uses $\delta_b = \frac{1}{2}(\delta_i + \delta_j)$ in (17). The nodal damage projection is

$$d_i = 1 - \frac{\text{intact bonds of } i}{\text{family size of } i} \in [0, 1]. \quad (18)$$

7.6 pddo_dynamic_frac – implicit dynamic fracture

Solves $\rho \ddot{\mathbf{u}} = \mathbf{L}(\mathbf{u})$ (no body force in the public kernel) with an implicit backward-Euler integrator embedded in the residual:

$$\mathbf{R}_i = \mathbf{L}(\mathbf{u})|_i - \rho \frac{\mathbf{u}_i - \mathbf{u}_i^{n-1} - \mathbf{v}_i^{n-1} \Delta t}{\Delta t^2}, \quad (19)$$

which Newton drives to zero each step; on convergence the velocity is committed as $\mathbf{v}^n = (\mathbf{u}^n - \mathbf{u}^{n-1})/\Delta t$. Bond failure is evaluated once per *converged* step on the committed displacement (never inside the Newton loop), so the tangent stays consistent within a step.

Table 15: pddo_dynamic_frac / explicit_pddo_frac parameters.

Key	Type	Constraint	Meaning
E	number	> 0 , required	Young's modulus
nu	number	$\in (-1, 0.5)$, required	Poisson's ratio
rho	number	> 0 , required	mass density
Gc / G0	number	> 0 , exactly one	critical energy release rate (alias pair)
state	string	required in 2-D, rejected in 3-D	"plane_stress" or "plane_strain"
tension_only	bool	required	break bonds only in tension
damage	bool	required	master switch for bond breaking

Projections: `strain`, `stress` (symmetric tensors, computed from PDDO displacement gradients and the elastic law), `vonMises` (scalar), `damage` (scalar).

7.7 explicit_pddo_frac – explicit dynamic fracture

The same physics as Section 7.6 integrated with the central-difference scheme

$$\mathbf{u}^{n+1} = 2\mathbf{u}^n - \mathbf{u}^{n-1} + \Delta t^2 \frac{\mathbf{L}(\mathbf{u}^n)}{\rho}. \quad (20)$$

No linear system is ever assembled – the input must not contain `LinearSolver` or `NonlinearSolver` blocks, and `pdtraction` boundary conditions are rejected (Section 6.6). Parameters are identical to Table 15. Additional characteristics:

- **Stability.** The step must satisfy a CFL-type bound; the kernel estimates $\Delta t \lesssim \Delta x / \sqrt{C_{11}/\rho}$ from the grid spacing and warns when exceeded.
- **Velocity ICs.** `"field": "velocity"` initial conditions are supported (only by this kernel) and seed the $\mathbf{u}^{-1}$ history so the first step moves with the prescribed velocity.

- **Damage update.** Bond breaking is evaluated every step on the current displacement.
- **Performance.** Per-bond operator weights are cached in flat compressed arrays at start-up (with an overflow guard for clouds beyond 2^{31} bonds). When compiled with `CUDA_ASSEMBLE` and `"assemble": "cuda"`, the bond loop runs on the GPU; if the GPU path fails it falls back to the CPU *permanently* for the rest of the run, so the irreversible damage state never diverges between the two code paths.

Projections: `damage`, `vonMises`, `stress`, `strain`.

7.8 `stress_cahnhilliard` – chemo-mechanics

Couples Cahn–Hilliard transport (Section 7.4) to linear elasticity through the composition eigen-strain $\boldsymbol{\varepsilon}^{\text{ch}} = \frac{\Omega}{3}(c - c_{\text{ref}})\mathbf{I}$ and the hydrostatic-stress term in the chemical potential:

$$\frac{\partial c}{\partial t} = \nabla \cdot [M(c) \nabla \mu], \quad M(c) = D c(1 - c), \quad (21)$$

$$\mu = \ln \frac{c}{1 - c} + \chi(1 - 2c) - \Omega \sigma_h - \kappa \nabla^2 c, \quad (22)$$

$$\boldsymbol{\sigma} = \mathbb{C} : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^{\text{ch}}), \quad \nabla \cdot \boldsymbol{\sigma} = \rho \ddot{\mathbf{u}} \quad (\text{or } \mathbf{0} \text{ if } \rho = 0). \quad (23)$$

The hydrostatic stress is evaluated in closed form,

$$\sigma_h = K_h \text{tr } \boldsymbol{\varepsilon} + C_h (c - c_{\text{ref}}), \quad (24)$$

with coefficients that depend on the plane reduction:

$$\begin{aligned} \text{plane stress:} \quad A &= \frac{E \Omega}{3(1 - \nu)}, \quad K_h = \frac{C_{11} + C_{12}}{3}, \quad C_h = -\frac{2A}{3}, \\ \text{plane strain / 3-D:} \quad A &= \frac{E \Omega}{3(1 - 2\nu)}, \quad K_h = \frac{C_{11} + 2C_{12}}{3} = K, \quad C_h = -A, \end{aligned} \quad (25)$$

where A is also the eigenstress modulus that enters the momentum residual as the body-force-like term $-A \nabla c$. The species transport uses the same conservative antisymmetric flux, ghost-bond exclusion and Laplacian calibration as the plain Cahn–Hilliard kernel. With $\rho > 0$ a backward-Euler inertia term is added to the momentum residual exactly as in Section 7.6; with $\rho = 0$ (or omitted) the mechanics are quasi-static and the displacements must be pinned somewhere to remove rigid-body modes.

Table 16: `stress_cahnhilliard` parameters.

Key	Type	Constraint	Meaning
<code>E</code>	number	> 0 , required	Young's modulus
<code>nu</code>	number	$\in (-1, 0.5)$, required	Poisson's ratio
<code>D</code>	number	> 0 , required	mobility scale in $M(c) = D c(1 - c)$
<code>Omega</code>	number	required	partial molar volume Ω
<code>cref</code>	number	required	stress-free reference concentration
<code>chi</code>	number	required	Flory interaction parameter
<code>kappa</code>	number	> 0 , required	gradient-energy coefficient
<code>rho</code>	number	≥ 0 , optional (default 0)	density; > 0 adds backward-Euler inertia
<code>state</code>	string	required in 2-D, rejected in 3-D	"plane_stress" or "plane_strain"

Projections: `strain`, `stress`, `vonMises`, `sigmaH` (σ_h), `fPrime`, `fDoublePrime`, `gradC` (∇c).

7.9 `frac_stress_cahnhilliard` – chemo-mechanics with fracture

Extends Section 7.8 with critical-stretch bond breaking. The design principle: *damage degrades mechanics only*. Bond health μ_{ij} multiplies

- the elastic stiffness contributions (16),
 - the eigenstress body force $-A \nabla c$, and
 - the $\Omega K_h \nabla \cdot \mathbf{u}$ mechanical-coupling term in the μ -equation,
- while species diffusion and the κ interface term are *never* degraded – a cracked region still transports species. Two fracture-specific refinements:

- **Swelling-corrected stretch.** The failure criterion uses the elastic part of the bond stretch,

$$s_{\text{el}} = s - \frac{\Omega}{3}(\bar{c}_{ij} - c_{\text{ref}}), \quad \bar{c}_{ij} = \frac{1}{2}(c_i + c_j), \quad (26)$$

so homogeneous swelling does not spuriously break bonds.

- **Residual stiffness.** A node whose bonds have all failed would leave a singular block in the Jacobian; the stiffness gate is therefore $g_{\text{eff}} = \max(\mu_{ij}, k_{\text{res}})$ applied through the `residual_stiffness` floor $k_{\text{res}} \in (0, 1]$ (default 10^{-8}).

Table 17: `frac_stress_cahnhilliard` parameters (in addition to every key of `stress_cahnhilliard`; here `rho` is *required*).

Key	Type	Constraint	Meaning
<code>rho</code>	number	≥ 0 , <i>required</i>	0: quasi-static (pin rigid modes!); > 0 : backward-Euler inertia
<code>Gc / G0</code>	number	> 0 , exactly one	critical energy release rate (alias pair)
<code>tension_only</code>	bool	required	break bonds only in tension
<code>damage_on / damage</code>	bool	exactly one	master switch for bond breaking (alias pair)
<code>residual_stiffness</code>	number	$\in (0, 1]$, default 10^{-8}	stiffness floor for fully severed nodes

Projections: those of `stress_cahnhilliard` plus damage.

8 Boundary Conditions

8.1 The BCSys`tem` block

BCSystem is a non-empty object; every entry is one boundary condition, named by a free label. Each entry requires `type` (one of `dirichlet`, `neumann`, `speciesflux`, `pdtraction`, `sdtraction`) and `phygroup` – the node group it acts on (Section 5.4). In a nonlocal method a “boundary condition” is a recipe for filling the rows of the fictitious ghost points, so `phygroup` is usually a `*nodes_ghost` group.

Two run-time safety nets complement the schema: a Dirichlet condition that pins a Cahn–Hilliard species DoF (`c` or `mu`) on boundary ghosts is rejected (the species flux drops ghost bonds, so such a pin is inert and would silently seal the domain), and any ghost wall left without *any* boundary condition is reported at start-up, because unconstrained one-sided PD rows are the most common cause of solver failure on refined meshes.

8.2 `dirichlet` – essential conditions

On a ghost group the default is the *mirror (reflection)* form: for each ghost g with mirror bulk partner b the row enforces

$$u_g + u_b = 2g(t) \quad \Longleftrightarrow \quad u|_{\text{wall}} = g(t), \quad (27)$$

i.e. the wall value midway between ghost and bulk equals the prescribed value exactly, second-order accurately. With `"direct": true`, on non-ghost groups, or for ghosts without a mirror partner, the row degenerates to the direct pin $u_g = g(t)$. Row replacement scales the unit diagonal to the mean magnitude of the matrix diagonal, preserving the conditioning of the assembled system.

Table 18: `dirichlet` keys.

Key	Type	Default	Meaning
<code>phygroup</code>	string	required	target node group
<code>dofs</code>	array	required	non-empty list of declared DoF names
<code>value</code>	number or array	required	prescribed value g_0 ; an array must match <code>dofs</code> in length
<code>velocity</code>	number or array	0	ramp rate: $g(t) = g_0 + \dot{g} t$
<code>box</code>	object	none	axis-aligned filter <code>{xmin,xmax,ymin,ymax,zmin,zmax}</code> ; only nodes inside all given bounds are affected (a selection that matches no node warns once)
<code>direct</code>	bool	false	force direct pinning (see below)

8.3 `neumann` – homogeneous natural conditions

Keys: `phygroup`, `dofs`, `value` – and the value *must be zero* (scalar 0 or an all-zero array); the public generic Neumann condition is homogeneous. It enforces the zero-normal-gradient mirror $u_g = u_b$. For inhomogeneous loads use `pdtraction`/`sdtraction` (mechanics) or `speciesflux` (species transport).

8.4 `speciesflux` – prescribed species influx

Keys: `phygroup`, `dofs` (must be exactly `["c"]`), `value` (the flux j). Requires a species kernel. The implementation is a *source form*: the ghost row keeps the zero-gradient mirror $c_g = c_b$ (so the PDDO family of the wall sees no artificial gradient), and the flux enters the mass balance of the near-wall *bulk* row as a source

$$R_b = -\frac{j}{h}, \quad (28)$$

where h is the wall-layer thickness (cell volume divided by face area where available, otherwise the ghost–bulk distance). Positive `value` means influx: the total species content grows at exactly j per unit wall area and time.

8.5 `pdtraction` and `sdtraction` – strong-form tractions

These conditions replace the ghost rows with the strong-form traction statement

$$\boldsymbol{\sigma}(\mathbf{x}_w) \cdot \mathbf{n} = \bar{\mathbf{t}}, \quad \boldsymbol{\sigma} = \mathbb{C} : \boldsymbol{\varepsilon} - \underbrace{A(c - c_{\text{ref}})}_{\text{sdtraction only}} \mathbf{I}, \quad (29)$$

built from the PDDO first-derivative operators of the ghost point; $\mathbf{n}$ is the dominant-axis outward normal of the wall. They require an assembled linear system and are therefore invalid with the explicit kernel. `pdtraction` is the purely mechanical form; `sdtraction` additionally subtracts the swelling eigenstress, which is the correct traction-free condition for the chemo-mechanical kernels.

Table 19: `pdtraction` / `sdtraction` keys.

Key	Type	<code>pdtraction</code>	<code>sdtraction</code>
<code>phygroup</code>	string	required	required
<code>E, nu</code>	numbers	required ($E > 0$, $\nu \in (-1, 0.5)$)	same
<code>state</code>	string	required in 2-D	optional in 2-D (default <code>plane_stress</code>); rejected in 3-D for both
<code>value</code>	array[dim]	required	optional (default zero traction)
<code>dofs</code>	array	optional, but <i>required</i> for the coupled kernels; must be <code>["ux", "uy", "uz"]</code>	
<code>Omega</code>	number	chemical coupling: <code>pdtraction</code> needs <code>Omega+c_ref+c_dof</code> together;	
<code>c_ref</code>	number	<code>sdtraction</code> needs <code>Omega</code> only (<code>c_ref</code> defaults to 0, <code>c_dof</code> to <code>"c"</code>)	
<code>c_dof</code>	string	must name the declared concentration field <code>"c"</code>	

The elastic constants of the traction condition are given explicitly (**E**, **nu**, **state**) so the boundary operator is self-contained; in practice they repeat the element parameters.

9 Initial Conditions

ICSystem is a non-empty object of named entries. Every entry requires **type** and targets either one DoF (**"dof": "c"**) or several (**"dofs": [...]**) – exactly one of the two spellings. Common optional keys:

- **field**: "solution" (default) or "velocity"; velocity ICs are accepted only with **explicit_pddo_frac**;
- **phgroup**: restrict to a node group (default: all bulk nodes).

Entries are applied in declaration order; later entries overwrite earlier ones on overlapping nodes. Keys of kind “scalar or array” accept one number (applied to every listed DoF) or an array matching **dofs** in length.

Table 20: IC generators. “s/a” = scalar or per-DoF array.

type	Keys	Field value at x
constant	value (s/a)	v
linear	offset (s/a, required), slope_x , slope_y , slope_z (s/a, default 0)	$v_0 + s_x x + s_y y + s_z z$
box	min_corner , max_corner (2–3 numbers, min $\leq$ max), inside_values , outside_values (s/a)	inside value within the box, outside value elsewhere
circle	center (2–3 numbers), radius (> 0), dr (≥ 0 , required), inside_values , outside_values	disc (2-D) / sphere (3-D); $dr = 0$ hard step, $dr > 0$ blends inside $\rightarrow$ outside over $[\text{radius}, \text{radius}+dr]$ with a C^1 cubic
ellipse	center , semi_axes (> 0 each), inside_values , outside_values , smoothness (≥ 0 , default 0)	ellipse (ellipsoid) indicator; smoothness = 0 hard step, > 0 a tanh blend across the level set
gaussian	center , sigma (> 0 each, per axis), amplitude (s/a), offset (s/a, default 0)	$v_0 + a \exp[-\sum_k (x_k - \mu_k)^2 / (2\sigma_k^2)]$
cosine	wavenumber (per axis), phase (per axis, default 0), amplitude (s/a), offset (s/a)	$v_0 + a \prod_k \cos(k_k x_k + \varphi_k)$
random	min , max (min $<$ max), seed (required, non-negative integer)	i.i.d. uniform values in $[\text{min}, \text{max}]$ from a seeded mt19937_64 – runs are bit-reproducible

A typical spinodal-decomposition start is a seeded random perturbation about the mean composition:

```
"ICSystem": {
  "noise": {"type": "random", "dof": "c",
            "min": 0.49, "max": 0.51, "seed": 12345}
}
```

10 Job Control, Time Stepping and Solvers

10.1 JobSystem

Optional; when omitted the job is static with serial assembly. When present, **type** is required.

10.2 Run drivers

The problem driver picks one of four run modes:

- **static** + implicit kernel: a single Newton solve, then one result save;

Table 21: JobSystem keys.

Key	Type	Default	Meaning
type	string	"static"	"static" (one solve) or "transient" (time loop; requires TimeStepping)
assemble	string	"serial"	"serial", "openmp" (needs PARALLEL_ASSEMBLE; thread count from OMP_NUM_THREADS) or "cuda" (needs CUDA_ASSEMBLE)

- **transient** + implicit kernel: the backward-Euler time loop of Section 10.3, one Newton solve per step;
- **transient** + **explicit_pddo_frac**: the matrix-free central-difference loop (Section 7.7);
- quasi-static matrix-free problems use an ADR (adaptive dynamic relaxation) driver – not reachable with the published kernel set, listed for completeness.

10.3 TimeStepping

Required for transient jobs, forbidden for static ones.

Table 22: TimeStepping keys.

Key	Type	Default	Meaning
dt	number	required	initial (and, non-adaptive, constant) step; > 0
total_time	number	required	end time; > 0
verbose	bool	true	per-step progress lines
adaptive	bool	false	enable adaptive step control
optimal_iters	int	5	Newton-iteration target of the controller; ≥ 1
growth_factor	number	1.25	step multiplier after an easy step; ≥ 1
cutback_factor	number	0.5	step multiplier after a failed step; $\in (0, 1)$
max_cutbacks	int	4	retries per step before giving up; ≥ 0
min_dt	number	0	lower bound on the step
max_dt	number	0	upper bound; 0 means “capped at the initial dt”

The adaptive controller works per step: if Newton *fails*, the step is retried with $\Delta t \leftarrow \Delta t \cdot \text{cutback_factor}$, up to **max_cutbacks** times, then the run aborts. If Newton *converges*, the next step grows by **growth_factor** when the iteration count was at most **optimal_iters** and shrinks by **cutback_factor** otherwise, always clamped to $[\text{min_dt}, \text{max_dt}]$. The last step is shortened to land exactly on **total_time**.

10.4 NonlinearSolver

Newton’s method without line search: assemble $\mathbf{K} \Delta \mathbf{U} = -\mathbf{R}$, apply boundary conditions, solve, update $\mathbf{U} += \Delta \mathbf{U}$. Convergence when $\|\mathbf{R}\| < \text{abs_tol}$ or $\|\mathbf{R}\| \leq \text{rel_tol} \cdot \|\mathbf{R}_0\|$.

Table 23: NonlinearSolver keys.

Key	Type	Default	Meaning
max_iters	int	50	maximum Newton iterations per solve; ≥ 1
abs_tol	number	10^{-9}	absolute residual-norm tolerance; > 0
rel_tol	number	10^{-12}	relative residual-norm tolerance; > 0
verbose	bool	true	per-iteration residual lines

10.5 LinearSolver

The block itself is optional (the built-in direct solver is the default). The AMGCL backend detects the block size (1–5 coupled DoFs per node) automatically and climbs a preconditioner ladder –

Table 24: Linear solver backends. Only the listed `params` keys are public; anything else is rejected.

type	Availability	Description and params
default	always	built-in direct profile (skyline) LDU factorisation without pivoting, with reverse Cuthill–McKee reordering to compress the profile. No parameters (<code>params</code> must be empty/omitted).
pardiso	USE_ONEAPI builds	Intel MKL PARDISO sparse direct solver. Params: <code>msglvl</code> (int ≥ 0 , default 0) – PARDISO statistics verbosity.
cusss	USE_CUDSS builds	NVIDIA cuDSS sparse direct solver on the GPU. No public parameters.
amgcl	USE_AMGCL builds	AMGCL BiCGSTAB with a preconditioner ladder (see below). Params: <code>tol</code> (> 0 , default 10^{-10}), <code>maxiter</code> (int ≥ 1 , default 200), <code>verbose</code> (bool, default <code>false</code>), <code>iluk_level</code> (int 1–6, default 2), <code>solver_threads</code> (int ≥ 0 , default 0 = automatic).

AMG-with-ILU(0) relaxation first, plain ILU(0) next, ILU(k) with `iluk_level` fill last – restarting the Krylov solve when a cheaper preconditioner stalls. Requesting `pardiso`, `cusss` or `amgcl` in a binary compiled without the corresponding CMake option is a start-up error.

11 Output System

11.1 The OutputSystem block

Table 25: OutputSystem keys.

Key	Type	Default	Meaning
interval	int	1	save every n -th step (≥ 1); step 0 and the final step are always saved
Fields	array	[]	projection names to write in addition to the solution; duplicates are rejected; "solution" is accepted and ignored (the solution is always written); a name no kernel provides is a fatal error
format	string	"vtu"	"vtu", "exodus" or "both"

Static jobs save once after the solve. The available projection names per kernel are listed in Section 7; vector and tensor projections are written with their natural number of components.

11.2 VTK output

Two files (per saved step) plus their ParaView index:

- `<stem>-PD-results[-NNNN].vtu`: the PD point cloud as vertex cells, with point data `ElmtID`, one scalar per DoF (canonical names `u`, `c`, `mu`, `ux`, ...) and the requested projections;
- `<stem>-Element-results[-NNNN].vtu`: the background mesh with the same data as cell data (one PD point per cell), convenient for contour plots;
- `<stem>-PD-results.pvd` / `<stem>-Element-results.pvd`: time-series indexes, rewritten at every save so a running simulation can be opened at any moment.

Transient file names carry a five-digit zero-padded step counter; static results have no counter.

11.3 ExodusII output

With "format": "exodus" (or "both") PeriX writes `<stem>.e` – a self-contained ExodusII database in CDF-2 format produced by the built-in writer; *no* NetCDF or SEACAS libraries are needed at build or run time. The mesh is written once with the proper element types (BAR2, TRI3, QUAD4, TETRA4, HEX8); every save appends a time plane with strictly increasing times. Each DoF becomes an element variable under its canonical name; multi-component projections are split into `name_1`, `name_2`, ... components.

12 Worked Examples

The `examples/` directory holds eleven decks covering every kernel, every mesh type, every solver backend and both assembly accelerators. Each deck is schema-validated by the test suite. Run any of them with

```
cd examples
../build/perix -i poisson.json
```

12.1 poisson.json – steady conduction

A 1000×1000 unit square (10^6 PD points), horizon factor 3.0, order 2, conductivity $\sigma = 1$, no source. Dirichlet walls $u = 0$ (left) and $u = 1$ (right) through the mirror form on the ghost groups; top and bottom carry no condition. OpenMP assembly, PARDISO solver, one static solve. The complete deck:

```
{
  "Mesh": {"type": "perix", "dim": 2, "xmax": 1.0, "ymax": 1.0,
    "nx": 1000, "ny": 1000, "meshtype": "quad4",
    "savemesh": false },
  "PDMesh": {"HorizonRadiusFactor": 3.0, "Order": 2},
  "DOFs": ["u" ],
  "ElmtSystem": {
    "poisson": {"type": "poisson",
      "params": {"sigma": 1.0, "f": 0.0 }}
  },
  "BCSystem": {
    "left": {"type": "dirichlet", "phygroup": "leftnodes_ghost",
      "dofs": ["u"], "value": 0.0 },
    "right": {"type": "dirichlet", "phygroup": "rightnodes_ghost",
      "dofs": ["u"], "value": 1.0 }
  },
  "JobSystem": {"type": "static", "assemble": "openmp" },
  "LinearSolver": {"type": "pardiso" }
}
```

The solution is the linear ramp $u = x$; a useful first check of any build.

12.2 poisson_amgcl.json and poisson_gpu.json

The same physics on a 50×50 grid, demonstrating the optional backends: `poisson_amgcl.json` solves with "type": "amgcl" (tol 10^{-10} , maxiter 200) and serial assembly; `poisson_gpu.json` uses "assemble": "cuda" with the cudss direct solver. They require binaries configured with `USE_AMGCL` and `CUDA_ASSEMBLE+USE_CUDSS` respectively.

12.3 diffusion.json – transient diffusion

A 500×500 unit square, $D = 0.01$, c initially zero, Dirichlet $c = 1$ at the bottom wall and $c = 0$ at the top, zero-flux Neumann on the sides. Backward Euler with fixed $\Delta t = 0.5$ to $t = 50$; results every 10 steps. The concentration front diffuses upward towards the steady linear profile.

12.4 spinodal.json – Cahn–Hilliard phase separation

A 250×250 unit square with $\chi = 3$ (deep quench), $\kappa = 10^{-3}$, $M_0 = 1$. The initial state is the seeded random perturbation $c \in [0.49, 0.51]$ (seed 12345, bit-reproducible) with $\mu = 0$; all four walls are homogeneous Neumann. The adaptive controller (initial $\Delta t = 10^{-3}$, `optimal_iters` 10, growth 1.2, cutback 0.5, `max_cutbacks` 6, Δt clamped to $[10^{-4}, 1]$) rides the stiffness of early spinodal decomposition and accelerates through late-stage coarsening to $t = 100$. Fields `fPrime` and `fDoublePrime` are written every 20 steps; total mass $\sum_i V_i c_i$ is conserved to solver tolerance.

12.5 kalthoff_winkler.json – explicit dynamic fracture

The classic Kalthoff–Winkler impact test: a pre-notched steel plate (Gmsh mesh `impact2D.msh`) struck between the notches. Maraging-steel parameters $E = 190$ GPa, $\nu = 0.25$, $\rho = 7800$ kg/m³, $G_0 = 69\,000$ J/m², plane stress, tension-only failure. The two notches are explicit `Cracks` segments at $x = 0.075$ and $x = 0.125$ (from $y = 0.05$ to $y = 0.1$) with `force_only` treatment. The impactor region (`impactnodes_ghost`) is driven by a direct Dirichlet condition with `velocity` $[0, -22]$ m/s; the remaining boundary (`freenodes_ghost`) is traction-free via homogeneous Neumann. Central-difference integration with $\Delta t = 10^{-8}$ s to $t = 10^{-4}$ s; `damage`, `vonMises` and `stress` saved every 1000 steps. The simulation reproduces the experimentally observed crack branches at roughly 70° to the notch line.

`kalthoff_winkler_gpu.json` is the same deck with `"assemble": "cuda"` – the entire bond loop runs on the GPU.

12.6 kalthoff_winkler_3D.json – 3-D explicit fracture

The 3-D extension on `impact3D.msh` with DoFs `ux,uy,uz` and `Gc` spelled directly (no `state` in 3-D). The notches are `center_crack` presets with `normal` $[1, 0, 0]$, `length` 0.05 and `width` 0.009 – planar cracks through the plate thickness. The out-of-plane displacement `uz` is pinned on `alldomain` to keep the thin plate in a plane-strain-like state, and the impactor drives `velocity` $[0, -22, 0]$. $\Delta t = 5 \times 10^{-8}$ s, output every 100 steps.

12.7 tensile_plate.json – implicit quasi-static fracture

A 1×0.4 plate (300×120 cells) with an edge crack from (0, 0.2) to (0.5, 0.2), loaded by `pdtraction` conditions: ± 0.0153884 vertical traction on the top/bottom walls and traction-free left/right walls (all with $E = 79.1403$, $\nu = 0.333$, plane stress – a normalised material set, $\rho = 268.198$, $G_c = 1.48388 \times 10^{-6}$). Note that the traction conditions address the *bulk* wall groups (`topnodes`, ...); the condition automatically maps each bulk node to its mirror ghost and writes the traction rows there. The implicit `pddo_dynamic_frac` kernel advances with the adaptive controller (Δt from 10^{-3} in $[10^{-4}, 10^{-2}]$, growth 1.2, cutback 0.5) to $t = 5$; the crack propagates across the ligament and the plate tears in two.

12.8 silicon.json – lithiation of a silicon disc

Stress-assisted Cahn–Hilliard on the `circle` mesh (radius 1, 400 layers, `VariableHorizon` on): a normalised model of Li insertion into an amorphous-Si particle. Parameters $E = 87.9336$, $\nu = 0.29$, $D = 10^{-4}$, $\Omega = 3$, $c_{\text{ref}} = 0$, $\chi = 2.6$, $\kappa = 2.19834 \times 10^{-6}$, $\rho = 2.41817 \times 10^{-22}$ (inertia negligible but nonzero), plane stress. Boundary conditions on the rim (`outernodes_ghost`): a constant species influx $j = 0.013889$ (`speciesflux`), zero-gradient μ , and a swelling-aware traction-free surface (`sdtraction` with the same E, ν, Ω). A small box around the centre pins `ux,uy` to remove rigid-body modes. Adaptive stepping to $t = 30$; `stress` and `sigmaH` written every 50 steps. The concentration front sweeps inward while the core is pushed into hydrostatic compression – the classical two-phase lithiation picture.

12.9 silicon_frac.json – lithiation with a pre-crack

The fracture variant: `frac_stress_cahnhilliard` with $G_c = 0.010662$, `tension_only` true, faster transport ($D = 0.03$) and a radial pre-crack from $(-2, 0)$ to $(-0.5, 0)$ – the segment reaches beyond the disc so the crack terminates exactly at the rim. With `"damage_on": false` the bond network stays frozen: the deck isolates the effect of the pre-crack (stress concentration, locally re-routed swelling stresses) from crack *growth*, which can be enabled by flipping the switch to true.

13 Testing

The test tree registers 29 CTest cases under four (overlapping) labels:

- **unit** (17 tests): one test executable per subsystem – every element kernel, the BC/IC systems, the circle-mesh anchor, the Exodus writer, initial velocities, the job system, `MeshModify`, the output system, the input schema, and the AMGCL solver components;
- **examples** (13): the input-schema suite plus a `ReadOnly_<deck>` validation of each of the eleven bundled decks (`perix -i deck --read-only`), so every shipped input is guaranteed to parse and validate;
- **integration** (1): `Run_poisson_amgcl`, a full end-to-end solve of the AMGCL Poisson deck with checked results;
- **amgcl** (2): the AMGCL unit suite and the end-to-end run (present only in `USE_AMGCL` builds).

```
cd build && ctest -j 16 --output-on-failure
```

14 License and Citation

PeriX is distributed under the GNU General Public License v3.0. The PDDO method is due to Madenci and co-workers; the standard references are:

- E. Madenci, A. Barut, M. Futch, *Peridynamic differential operator and its applications*, Computer Methods in Applied Mechanics and Engineering 304 (2016) 408–451.
- E. Madenci, A. Barut, M. Dorduncu, *Peridynamic Differential Operator for Numerical Analysis*, Springer, 2019.

Please cite the repository (<https://github.com/MatMechLab/PeriX>) when using PeriX in published work.